PROJECT 10

COURSE REGISTRATION SYSTEM

Business Case Study | Python Object-Oriented Programming

A beginner-friendly application concept for managing student records, course details, enrolments and course withdrawals, with data saved between sessions using JSON files.

Project type	Menu-driven console application
Primary users	Course coordinators and administrators in a proposed small-institute use case
Technology	Python, OOP, lists, dictionaries, loops and JSON
Data storage	Local JSON files: students.json and courses.json

Prepared as a portfolio project

1. Executive Summary

The Course Registration System is a small Python application designed to demonstrate how a basic course-enrolment process can be organised digitally. It allows an operator to add students, create courses, register students for courses, drop registrations and view records. Student and course information is stored in JSON files so the data can be loaded again when the program is restarted.

Business idea: Replace scattered manual record-keeping with one simple workflow. This project is an educational prototype, not a production-ready student information system.

2. Business Problem

In a small training centre or educational institute, course and enrolment information may be tracked using paper forms, notebooks or separate spreadsheets. This can make routine tasks harder to manage when staff need to confirm enrolments or update a withdrawal.

Potential challenges in a manual process

• Student details may be recorded in different places.
• Course information and fees may be difficult to locate quickly.
• Duplicate enrolments may occur if records are not checked consistently.
• Dropped courses may not be reflected in every record.
• Records may need to be re-entered when a working session ends.

3. Proposed Solution

The application provides a menu-driven way to perform core registration tasks. The operator selects an option, enters the required identifiers and details, and receives a confirmation or validation message. Student IDs and course IDs connect each student to their registered courses.

4. Project Objectives

• Maintain student records with unique student IDs.
• Maintain course records with course IDs, names and fees.
• Register a student for an existing course.
• Prevent duplicate registration for the same course.
• Allow a student to drop a registered course.
• Display student, course and registration information.
• Save and reload records using JSON files.

5. Users and Stakeholders

Stakeholder	Interest / responsibility
Administrator / coordinator	Adds student and course records; manages registrations and withdrawals.
Student	Is represented by a student record and course enrolments.
Institute management	May use registration information to understand course participation.

6. Functional Scope

Feature	What it does
Add student	Accepts a student ID and name; checks that the ID is not already in use.
Add course	Accepts a course ID, name and fee; validates the fee input.
Display students	Lists saved students and the number of courses associated with each.
Display courses	Lists available courses and their fees.
Register student	Checks that both records exist and prevents the same course being registered twice.
Drop course	Removes a course from a student's registered-course list when present.
Display registrations	Shows each student's registered courses and corresponding course fees.
Save / load data	Writes records to JSON files and loads them when the program starts.

7. Typical Workflow

Step 1: Add a student with a unique ID.
Step 2: Add a course with a unique course ID and fee.
Step 3: Register the student using the student ID and course ID.
Step 4: View the registration to confirm enrolment.
Step 5: Drop the course if the student withdraws.
Step 6: The system saves changes to JSON files and loads them during the next run.

8. Data Design

The program uses dictionaries in memory to organise student and course objects. Each student object stores a list of course IDs. Those IDs connect the student record to the course records.

File	Information stored
students.json	Student ID, student name and registered course IDs.
courses.json	Course ID, course name and course fee.

9. Object-Oriented Programming Design

Student class: Represents a student and stores the student's ID, name and registered courses.
Course class: Represents a course and stores its ID, name and fee.
RegistrationSystem class: Coordinates the application. Its methods add and display records, manage registrations, and save or load JSON data.

This structure groups related data and behaviour together, making the code easier to read, reuse and maintain than a single collection of unrelated statements.

10. Expected Benefits

The prototype demonstrates how a simple digital workflow can make routine record management more organised. It can reduce repeated manual entry, provide basic validation, make registration details easier to display and preserve data between program sessions. These are expected benefits of the design; no measured time or cost savings are claimed.

11. Testing Plan

Test	Expected result
Add a new student	Student is added and saved.
Add an existing student ID	Duplicate ID is rejected.
Add a course with a valid fee	Course is added and saved.
Enter invalid fee text	The program displays a validation message.
Register for an existing course	Course ID is added to the student's registrations.
Register the same course twice	Duplicate registration is rejected.
Drop a registered course	Course ID is removed from the student's registrations.
Restart the application	Previously saved records are loaded from JSON.

12. Limitations and Future Improvements

Current limitations: This is a local, single-user console prototype. It does not include login, role-based permissions, a graphical interface, online access, a database server, course capacity limits, timetable conflict checks, payment processing or automated fee collection. JSON files are suitable for this learning project but are not ideal for concurrent multi-user use.

Possible future improvements:

- Add a graphical or web-based interface.
- Use SQLite or another database for structured storage.
- Add login and administrator/student roles.
- Add course capacity and waiting-list management.
- Generate enrolment summaries and reports.
- Add search, update and delete options for records.

13. Conclusion

The Course Registration System demonstrates the use of Python OOP to solve a practical record-management problem. By separating students, courses and registration operations into classes, and using JSON for persistent storage, the project brings together core programming concepts in one beginner-level application. It provides a foundation that could be expanded into a more complete course-management solution.

Portfolio note: This case study describes the implemented project scope and a proposed educational use case. Any operational benefits are expected outcomes, not measured results.